10. Outil de débogage des providers

Pour aider à déboguer vos providers, il est possible d'ajouter un observateur global qui va permettre de loguer les chargements, déchargements, mises à jour et erreurs des providers. Cela peut être très utile pour visualiser comment les providers interagissent entre eux et comment ils sont utilisés dans l'application.

Ajoutez la classe suivante dans votre projet :

lib/core/tools/my_provider_observer.dart

```dart
import 'package:flutter/foundation.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

final class MyProviderObserver extends ProviderObserver {
  String _info(ProviderObserverContext ctx) =>
      ctx.provider.name ?? ctx.provider.runtimeType.toString();

  @override
  void didUpdateProvider(
    ProviderObserverContext context,
    Object? previousValue,
    Object? newValue,
  ) {
    debugPrint('🔁 Provider updated: ${_info(context)}');
    debugPrint('➡️  New value: $newValue');
  }

  @override
  void providerDidFail(
    ProviderObserverContext context,
    Object error,
    StackTrace stackTrace,
  ) {
    debugPrint('🔁 Provider failed: ${_info(context)}');
    debugPrint('➡️  Error: $error');
    debugPrint('     $stackTrace');
  }

  @override
  void didAddProvider(ProviderObserverContext context, Object? value) {
    debugPrint('➕ Provider added: ${_info(context)}');
  }

  @override
  void didDisposeProvider(ProviderObserverContext context) {
    debugPrint('🗑️ Provider disposed: ${_info(context)}');
  }
}
```

Et modifiez le fichier `main.dart` pour ajouter cet observateur à votre `ProviderContainer` :

lib/main.dart

```
...

void main() async {
  ...
  runApp(
    ProviderScope(
      observers: [MyProviderObserver()],
      child: MyApp(),
    ),
  );
}
```

Puisqu'on est occupé à configurer les providers Riverpod de manière globale, profitons-en aussi pour désactiver le *retry* automatique des providers en cas d'erreur. En effet, depuis la version 3 de Riverpod, le *retry* est activé par défaut et peut provoquer des "boucles" inutiles si le provider ne se charge pas correctement (exception durant le `build`).

lib/main.dart

```
...

void main() async {
  ...
  runApp(
    ProviderScope(
      observers: [MyProviderObserver()],
      retry: (int _, Object _) => null,
      child: MyApp(),
    ),
  );
}
```